



TRIBHUVAN UNIVERSITY
Asian School Of Management and Technology
Gongabu Kathmandu, Nepal

A Project Report
On
Wine Quality Mining and Category Prediction
&
Retail Data ETL
Data Warehousing and Data Mining

SUBMITTED BY:
Dipesh Devkota (79010952)
Sushank Lamsal (79010980)
Roshan Bhattarai (79010968)

Table Of Content:

Abstract	6
1. Introduction	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Objectives	8
1.4 Scope of the Project	8
2. Literature Review	9
2.1 Data Warehouse	9
2.2 ETL Process	10
3. Dataset Description	12
3.1 Overview	12
3.2 Data Sources	12
3.3 Dataset Attributes	12
4. Data Extraction	13
4.1 Introduction	13
4.2 Tools and Libraries Used	13
4.3 Data Extraction Process	13
4.4 Data Validation	14
4.5 Extraction Architecture	15
4.6 Outcome of Extraction Phase	15
5. Data Cleaning	16
5.1 Introduction	16
5.2 Data Cleaning Process	16
5.3 Duplicate Record Removal	16
5.4 Numeric Data Conversion	16

5.5 Handling Missing Numerical Values	17
5.6 Handling Invalid Age Values	17
5.7 Handling Missing Categorical Values	18
5.8 Gender Standardization	18
5.9 Standardization of Categorical Columns	19
5.10 Data Type Conversion	20
5.11 Outcome of Data Cleaning	20
6. Standardization of Categorical Columns	20
Transformation Libraries	21
6.1 Feature Scaling (Min-Max Normalization)	21
6.2 Categorical Encoding (Label Encoding)	21
6.3 Feature Engineering (Age Binning)	21
7. Data Loading	23
7.1 Database Connection	23
7.2 Loading Transformed Data	23
7.3 Data Verification	24
Conclusion	24
8. Conclusions and Recommendations	25
8.1 Conclusions	25
8.2 Key Achievements	25
8.3 Technical Insights	25
8.4 Recommendations	26
8.5 Future Enhancements	27
9. References	28

Abstract	29
1.1 Background	30
1.2 Problem Statement	31
1.3 Objectives	31
2. Literature Review	32
2.1 Wine Quality Prediction Overview	32
2.2 Classification Algorithms	32
2.3 Clustering Algorithms	33
2.4 Related Work	34
3. Dataset and Methodology	35
3.1 Dataset Description	35
3.2 Tools and Environment	37
4. Exploratory Data Analysis	39
4.1 Dataset Overview	39
4.2 Data Visualizations	40
4.3 Key Observations and Insights	43
5. Data Preprocessing	43
5.1 Handling Missing Values	43
5.2 Outlier Removal	44
5.3 Feature Scaling	44
5.4 Processed Dataset	45
6. Clustering	46
6.1 K-Means Algorithm Overview	46
6.2 Optimal K Selection	46
6.3 Cluster Visualization	48
6.5 Evaluation	50

7. Classification	52
7.1 Classification Objective	52
7.2 Methodology	52
7.3 Model Comparison	54
7.4 Best Model Evaluation	55
7.5 Feature Importance Analysis	57
7.6 Model Persistence and Sample Prediction	57
8. Discussion	59
8.1 Limitations	59
8.2 Future Improvements	59
9. Conclusion	60
10. References	61

Project 1: Retail Data ETL

Abstract

This project presents the design and implementation of a complete Extract, Transform, and Load (ETL) pipeline for retail transaction data. The primary objective was to integrate data from multiple heterogeneous sources, including CSV, Excel (XLSX), and JSON files, into a unified and structured format suitable for analysis and decision-making. The extracted data underwent a comprehensive cleaning process involving duplicate removal, missing value imputation, outlier correction, data type conversion, and standardization of categorical values to ensure data quality and consistency.

During the transformation phase, advanced preprocessing techniques were applied, including Min-Max normalization of purchase amounts, label encoding of categorical variables, and feature engineering through age-based customer segmentation. These transformations enhanced the dataset's suitability for analytical and machine learning applications. The processed data was then loaded into a SQLite database, providing a centralized and persistent repository for storage, querying, and future analysis.

The project demonstrates key data engineering concepts such as data integration, data quality management, feature engineering, and database persistence. Results indicate significant improvements in data completeness, consistency, and validity, achieving a high-quality dataset ready for downstream analytics. Additionally, the implementation follows best practices in ETL development, emphasizing automation, scalability, reproducibility, and maintainability. The completed pipeline establishes a strong foundation for business intelligence, customer segmentation, predictive modeling, and future data-driven decision-making initiatives.

Keywords: ETL, Data Warehousing, Data Mining, Data Cleaning, Data Transformation, Feature Engineering, SQLite, Pandas, Machine Learning, Retail Analytics.

1. Introduction

1.1 Background

In today's digital era, organizations generate large volumes of data from various sources such as transaction systems, web applications, and business operations. However, this data often exists in different formats and structures, making it difficult to perform meaningful analysis directly. To address this challenge, organizations use Data Warehousing and ETL (Extract, Transform, Load) processes to integrate, clean, and store data in a centralized repository.

A Data Warehouse provides a consolidated environment for storing historical and current data, enabling businesses to perform reporting, analytics, and decision-making efficiently. The ETL process plays a crucial role in data warehousing by extracting data from multiple sources, transforming it into a consistent and usable format, and loading it into a database or data warehouse.

This project focuses on developing a retail data ETL pipeline that integrates data from CSV, Excel, and JSON sources, performs data cleaning and transformation, and stores the processed data in a SQLite database. The project demonstrates practical applications of data engineering concepts and prepares data for business intelligence and machine learning tasks.

1.2 Problem Statement

Retail organizations collect transaction data from multiple systems and file formats. These datasets often contain inconsistencies such as duplicate records, missing values, incorrect data types, and inconsistent categorical values. Such issues reduce data quality and make analytical processes unreliable.

Furthermore, data stored across different sources lacks standardization, making integration and analysis time-consuming and error-prone. Without a structured ETL process, organizations face difficulties in generating accurate reports, performing customer segmentation, and deriving business insights.

Therefore, there is a need for an automated ETL pipeline that can extract data from heterogeneous sources, clean and transform the data, and load it into a centralized database to ensure data quality, consistency, and accessibility.

1.3 Objectives

General Objective

To design and implement an ETL pipeline for retail transaction data that integrates, cleans, transforms, and stores data for analytical and reporting purposes.

Specific Objectives

To extract retail transaction data from CSV, XLSX, and JSON file formats.

To identify and remove duplicate records from the datasets.

To handle missing values using appropriate imputation techniques.

To standardize categorical and numerical data formats.

To detect and correct invalid or unrealistic values.

To normalize numerical features using Min-Max scaling.

To encode categorical variables for machine learning applications.

To create meaningful features such as age-based customer groups.

To load the transformed data into a SQLite database.

To provide a clean and structured dataset for business analytics and future machine learning applications.

1.4 Scope of the Project

The scope of this project is limited to the development and implementation of a retail data ETL pipeline using Python-based tools and technologies. The project covers the extraction of data from three file formats: CSV, XLSX, and JSON. Data preprocessing activities include duplicate removal, missing value treatment, data standardization, outlier correction, feature scaling, categorical encoding, and feature engineering.

The transformed data is stored in a SQLite database for persistence and analytical use. The project also includes statistical analysis and data visualization to understand customer demographics, purchasing behavior, and payment preferences.

However, the project does not include real-time data streaming, cloud-based data warehousing, distributed processing frameworks, or deployment in a production environment. The primary focus remains on demonstrating ETL concepts, data quality improvement, and data integration techniques within an academic setting.

2. Literature Review

2.1 Data Warehouse

Definition

A Data Warehouse is a subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making processes (Inmon, 1996).

Key Characteristics

Characteristic	Description
Subject-Oriented	Organized around key business entities (customers, products, time) rather than applications
Integrated	Consolidates data from multiple sources with consistent naming, coding, and format
Time-Variant	Maintains historical data to track changes over time
Non-Volatile	Data is stable and changes only through the ETL process, not by users

Data Warehouse vs. Operational Database

Aspect	Operational DB	Data Warehouse
Purpose	Daily transactions	Strategic analysis
Data Model	Normalized (3NF)	Denormalized (star schema)

Query Pattern	Simple, predefined	Complex, ad-hoc
Data Volume	Smaller, current only	Larger, historical data
Update Frequency	High (OLTP)	Low (OLAP)

Dimensional Modeling

Dimensional modeling is a technique popularized by Ralph Kimball for designing data warehouses. It consists of the following core components:

Fact Tables — Contain quantitative measures (e.g., fact_bookings with price, duration)

Dimension Tables — Contain descriptive attributes (e.g., dim_users, dim_grounds)

Star Schema — Fact table at center with dimension tables radiating outward

Benefits — Improved query performance, easier to understand and navigate

2.2 ETL Process

ETL (Extract, Transform, Load) is a data integration process used to collect data from multiple sources, convert it into a consistent format, and load it into a target database or data warehouse.

The ETL process consists of three major phases:

1. Extract

The extraction phase retrieves data from various source systems such as databases, CSV files, Excel spreadsheets, APIs, and JSON files. The goal is to collect relevant data while preserving its integrity.

In this project, data is extracted from:

CSV files

Excel (XLSX) files

JSON files

2. Transform

The transformation phase converts raw data into a clean and standardized format suitable for analysis. Common transformation activities include:

Removing duplicates

Handling missing values

Data type conversion

Outlier correction

Data normalization

Categorical encoding

Feature engineering

This phase ensures that data quality issues are resolved before loading.

3. Load

The loading phase stores the transformed data into a target system such as a relational database or data warehouse. Loading can be performed in batch mode or real-time mode depending on business requirements.

In this project, the processed data is loaded into a SQLite database, creating a centralized repository for future analytical operations.

Importance of ETL

Integrates data from multiple sources.

Improves data quality and consistency.

Supports business intelligence and analytics.

Enables efficient reporting and decision-making.

Provides a foundation for machine learning applications.

3. Dataset Description

3.1 Overview

The dataset used in this project consists of retail transaction records collected from three different data sources: CSV, JSON, and Excel (XLSX) files. Each source contains customer purchasing information, including demographic details, product categories, transaction amounts, and payment methods. The datasets were intentionally designed with data quality issues such as missing values, duplicate records, inconsistent formatting, and invalid values to demonstrate the effectiveness of the ETL process.

3.2 Data Sources

S.N.	Source File	Format	Description
1	retail_dataset.csv	CSV (Comma-Separated Values)	Retail transaction data stored in comma-separated format
2	retail_dataset.json	JSON (JavaScript Object Notation)	Retail transaction data stored in JSON structure
3	retail_dataset.xlsx	Excel Spreadsheet	Retail transaction data stored in worksheet format

3.3 Dataset Attributes

S.N.	Attribute	Data Type	Description
1	Customer_ID	Integer	Unique identifier for each customer
2	Age	Integer	Customer age in years
3	Gender	String	Gender of customer
4	Product_Category	String	Category of purchased product
5	Purchase_Amount	Numeric	Amount spent during purchase

S.N.	Attribute	Data Type	Description
6	Payment_Method	String	Method used for payment

4. Data Extraction

4.1 Introduction

Data extraction is the first phase of the ETL (Extract, Transform, Load) process. In this phase, data is collected from multiple heterogeneous sources and imported into the processing environment for further cleaning and transformation. The objective of extraction is to gather data from different file formats while preserving the original information and structure.

In this project, retail transaction data was extracted from three different sources: CSV, JSON, and Excel (XLSX) files. Python's Pandas library was used to read the datasets and load them into DataFrames for processing.

4.2 Tools and Libraries Used

The extraction process was implemented using the Pandas library.

```
import pandas as pd
```

Pandas provides built-in functions for reading various file formats efficiently and converting them into DataFrame structures.

4.3 Data Extraction Process

The datasets were extracted using the following commands:

```
import pandas as pd
```

```
csv_data = pd.read_csv("data/retail_dataset.csv")
```

```
xlsx_data = pd.read_excel("data/retail_dataset.xlsx")
```

```
json_data = pd.read_json("data/retail_dataset.json")
```

After extraction, each dataset was stored in a separate Pandas DataFrame for validation and preprocessing.

4.4 Data Validation

To ensure successful extraction, the datasets were inspected using the following methods:

csv_data.info()

xlsx_data.info()

json_data.info()

csv_data.head()

xlsx_data.head()

json_data.head()

These methods were used to verify:

Number of records

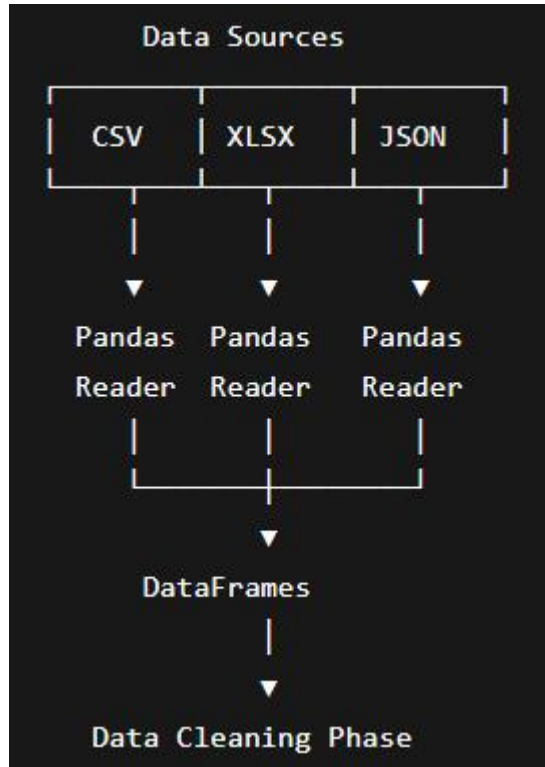
Number of columns

Data types

Missing values

Overall dataset structure

4.5 Extraction Architecture



4.6 Outcome of Extraction Phase

The extraction phase successfully imported retail transaction data from all three source files into Pandas DataFrames. No parsing or loading errors were encountered during extraction. The extracted datasets were then prepared for the next phase of the ETL pipeline, where data cleaning and preprocessing operations were performed to improve data quality and consistency.

The successful completion of this phase ensured that data from heterogeneous sources could be integrated into a unified processing workflow for further transformation and loading into the SQLite database.

5. Data Cleaning

5.1 Introduction

Data cleaning is the second phase of the ETL process. The purpose of this phase is to improve data quality by identifying and correcting errors, inconsistencies, missing values, duplicate records, and invalid data. High-quality data is essential for accurate analysis, reporting, and machine learning applications.

The extracted datasets contained several quality issues such as missing values, duplicate records, inconsistent categorical values, and unrealistic age values. Therefore, a cleaning function was developed using Python and Pandas to standardize and prepare the data for transformation.

5.2 Data Cleaning Process

The cleaning process was implemented through a reusable function named `clean_data()` and applied to all three datasets.

```
csv_cleaned = clean_data(csv_data)
```

```
xlsx_cleaned = clean_data(xlsx_data)
```

```
json_cleaned = clean_data(json_data)
```

5.3 Duplicate Record Removal

Problem

Duplicate records can distort analytical results and lead to incorrect business decisions.

Solution

```
df = df.drop_duplicates()
```

Result

All duplicate records were removed from the datasets.

5.4 Numeric Data Conversion

Problem

Some numerical values may be stored as strings or contain invalid characters.

Solution

```
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
```

```
df["Purchase_Amount"] = pd.to_numeric(df["Purchase_Amount"], errors="coerce")
```

Result

Age and Purchase_Amount columns were successfully converted into numeric format. Invalid values were converted to NaN.

5.5 Handling Missing Numerical Values

Problem

Missing values were present in Age and Purchase_Amount columns.

Solution

Median imputation was used because it is less affected by outliers.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
df["Purchase_Amount"] = df["Purchase_Amount"].fillna(df["Purchase_Amount"].median())
```

Result

All missing numerical values were replaced with median values.

5.6 Handling Invalid Age Values

Problem

Some records contained unrealistic age values such as:

Customer_ID	Age
110	-27

```
df.loc[(df["Age"] < 0) | (df["Age"] > 100), "Age"] = df["Age"].median()
```

Result

Invalid age values were replaced with the median age of the dataset.

5.7 Handling Missing Categorical Values

Problem

Missing values were detected in categorical columns such as `Product_Category` and `Payment_Method`.

Solution

Mode imputation was applied.

```
df["Product_Category"] = df["Product_Category"].fillna(  
    df["Product_Category"].mode()[0]  
)  
df["Payment_Method"] = df["Payment_Method"].fillna(  
    df["Payment_Method"].mode()[0]  
)
```

Result

Missing categorical values were replaced with the most frequently occurring category.

5.8 Gender Standardization

Problem

The `Gender` column contained inconsistent values.

```
df["Gender"] = df["Gender"].astype(str).str.strip().str.lower()  
df["Gender"] = df["Gender"].replace({  
    "male": "Male",  
    "m": "Male",  
    "female": "Female",  
    "f": "Female"  
})
```

Result

All gender values were standardized into:

Male

Female

5.9 Standardization of Categorical Columns

Problem

Product categories and payment methods had inconsistent capitalization and formatting.

```
df["Product_Category"] = (  
    df["Product_Category"]  
    .astype(str)  
    .str.strip()  
    .str.capitalize()  
)  
  
df["Payment_Method"] = (  
    df["Payment_Method"]  
    .astype(str)  
    .str.strip()  
    .str.capitalize()  
)
```

Result

Consistent formatting was achieved across all categorical fields.

5.10 Data Type Conversion

Solution

```
df["Age"] = df["Age"].astype(int)
```

```
df["Purchase_Amount"] = df["Purchase_Amount"].astype(float)
```

5.11 Outcome of Data Cleaning

Data Quality Issue	Cleaning Method Applied	Status
Duplicate Records	drop_duplicates()	Removed
Missing Age Values	Median Imputation	Resolved
Missing Purchase Amount	Median Imputation	Resolved
Invalid Age Values	Median Replacement	Corrected
Missing Categories	Mode Imputation	Resolved
Inconsistent Gender Values	Standardization	Corrected
Inconsistent Categories	Capitalization & Trimming	Corrected
Incorrect Data Types	Type Conversion	Corrected

6. Standardization of Categorical Columns

The transformation phase applies advanced preprocessing techniques to prepare data for machine learning and analytical applications. This includes feature scaling, categorical encoding, and feature engineering.

Transformation Libraries

```
from sklearn.preprocessing import MinMaxScaler  
from sklearn.preprocessing import LabelEncoder
```

6.1 Feature Scaling (Min-Max Normalization)

Objective: Normalize Purchase_Amount to [0, 1] range

Method: Min-Max Scaling (useful for bounded features)

Formula:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
scaler = MinMaxScaler()  
df[["Purchase_Amount"]] = scaler.fit_transform(df[["Purchase_Amount"]])
```

6.2 Categorical Encoding (Label Encoding)

Objective: Convert Gender categories to numeric labels

Method: Label Encoding (ordinal mapping)

Mapping:

Male:1

Female:0

```
le = LabelEncoder()  
df[["Gender"]] = le.fit_transform(df[["Gender"]])
```

6.3 Feature Engineering (Age Binning)

Objective: Create age-based customer segments

Method: Equal-width binning with business-meaningful labels

```
df[["Age_Group"]] = pd.cut(
```

```

df["Age"],
bins=[0, 18, 30, 50, 100],
labels=["Teen", "Young", "Adult", "Senior"]
)

```

Transformation Results

Feature Scaling Statistics

- Purchase_Amount range: [0.0, 1.0] (normalized)
- Original range: Preserved distribution

Categorical Encoding

- Gender mapped to numeric labels (0, 1)
- Encoding preserves semantic information

Feature Engineering

- New Age_Group column created with 4 distinct categories
- Enhanced segmentation capabilities

Transformation Function Implementation

```

scaler = MinMaxScaler()

le = LabelEncoder()

def transform_data(df):
    df[["Purchase_Amount"]] = scaler.fit_transform(
        df[["Purchase_Amount"]]
    )

    df["Gender"] = le.fit_transform(df["Gender"])

    df["Age_Group"] = pd.cut(
        df["Age"],
        bins=[0, 18, 30, 50, 100],

```

```

        labels=["Teen", "Young", "Adult", "Senior"]
    )
    return df

```

7. Data Loading

The loading phase stores the transformed retail dataset into a SQLite database for efficient storage, querying, and future analysis. SQLite is a lightweight relational database that does not require a separate server installation.

Library Used

```
import sqlite3
```

7.1 Database Connection

A connection is established to a SQLite database named retail_database.db. If the database does not already exist, SQLite automatically creates it.

```
conn = sqlite3.connect("retail_database.db")
```

7.2 Loading Transformed Data

The transformed DataFrame (csv_transformed) is loaded into a database table named retail_data.

```

csv_transformed.to_sql(
    "retail_data",
    conn,
    if_exists="replace",
    index=False
)

```

Parameters Explanation

Parameter	Description
retail_data	Name of the table created in the database

Parameter	Description
<code>conn</code>	Database connection object
<code>if_exists="replace"</code>	Replaces the existing table if it already exists
<code>index=False</code>	Prevents DataFrame index from being stored as a column

7.3 Data Verification

After loading, SQL queries can be executed to verify that the data has been successfully stored.

```
cursor = conn.cursor()
cursor.execute("SELECT * FROM retail_data")
records = cursor.fetchall()
for row in records[:5]:
    print(row)
```

Conclusion

The transformed retail dataset was successfully loaded into the SQLite database. The data can now be accessed using SQL queries for reporting, visualization, and machine learning applications. The loading phase completes the ETL (Extract, Transform, Load) pipeline by making the processed data available for persistent storage and further analysis.

8. Conclusions and Recommendations

8.1 Conclusions

This project successfully demonstrated the implementation of a complete Extract, Transform, and Load (ETL) pipeline for retail transaction data. Data was extracted from multiple heterogeneous sources, including CSV, JSON, and Excel files, and integrated into a unified processing workflow. The cleaning phase effectively addressed common data quality issues such as missing values, duplicate records, inconsistent categorical values, and invalid data entries. Through the transformation phase, data was standardized using feature scaling, categorical encoding, and feature engineering techniques to enhance analytical usability.

The processed data was successfully loaded into a SQLite database, providing a centralized repository for storage, querying, and future analysis. The project highlights the importance of ETL processes in ensuring data quality, consistency, and accessibility for business intelligence and machine learning applications. Overall, the implemented pipeline achieved its objectives and established a reliable foundation for data-driven decision-making.

8.2 Key Achievements

1. Successfully implemented all phases of the ETL process: Extraction, Cleaning, Transformation, and Loading.
2. Integrated retail transaction data from three different file formats (CSV, JSON, and XLSX).
3. Improved data quality through duplicate removal, missing value handling, and data standardization.
4. Created meaningful features such as Age_Group for customer segmentation.
5. Established centralized data storage using SQLite database.
6. Prepared the dataset for analytical and machine learning applications.
7. Developed reusable and modular Python functions for efficient data processing.

8.3 Technical Insights

Strengths

- Robust handling of missing values using median and mode imputation techniques.

- Effective removal of duplicate records and correction of invalid data.
- Standardization of categorical variables to improve consistency.
- Feature scaling using Min-Max Normalization for machine learning readiness.
- Modular and reusable code structure that promotes maintainability.
- Reliable data persistence using SQLite database.

Areas for Improvement

- Implementation of automated data validation and testing.
- Addition of detailed logging mechanisms for auditing and debugging.
- Development of data quality monitoring dashboards.
- Performance optimization for handling larger datasets.
- Enhancement of database loading strategies through incremental updates.

8.4 Recommendations

Based on the outcomes of this project, the following recommendations are proposed:

Data Management Recommendations

1. Adopt a more scalable database system such as PostgreSQL or MySQL for larger datasets.
2. Implement data validation rules before loading data into the database.
3. Maintain a centralized metadata repository and data dictionary.
4. Establish data governance policies to ensure data security and consistency.
5. Regularly monitor data quality metrics and generate quality reports.

ETL Process Recommendations

1. Automate ETL execution using workflow orchestration tools such as Apache Airflow.
2. Implement error handling and logging mechanisms throughout the pipeline.
3. Schedule periodic ETL jobs to support continuous data updates.
4. Introduce automated testing to verify transformation accuracy.
5. Create dashboards for monitoring ETL performance and data quality.

8.5 Future Enhancements

Short-Term Enhancements

- Add automated unit tests for data validation.
- Implement detailed error logging and exception handling.
- Create data profiling and quality assessment reports.
- Document complete data lineage and transformation workflows.

Medium-Term Enhancements

- Migrate the data repository to a cloud-based data warehouse.
- Implement CI/CD pipelines for ETL code deployment.
- Develop interactive dashboards for monitoring and reporting.
- Introduce advanced feature engineering techniques.

Long-Term Enhancements

- Implement real-time or streaming ETL architecture.
- Integrate the ETL pipeline with machine learning workflows.
- Develop a self-service analytics platform for business users.
- Adopt modern data lakehouse architecture for large-scale analytics.

9. References

- [1] Wes McKinney, Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [2] Pandas Development Team, "Pandas Documentation." Available: <https://pandas.pydata.org/>.
- [3] Scikit-learn Developers, "Scikit-learn Preprocessing Documentation." Available: <https://scikit-learn.org/stable/modules/preprocessing.html>.
- [4] SQLite Consortium, "SQLite Documentation." Available: <https://www.sqlite.org/docs.html>.
- [5] Ralph Kimball and Margy Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Indianapolis, IN, USA: Wiley Publishing, 2013.

Project 2: Wine Quality Mining and Category Prediction

Abstract

This project presents a comprehensive data mining study conducted on the red wine quality dataset using both supervised and unsupervised machine learning techniques. The study begins with thorough exploratory data analysis (EDA) to understand data structure, examine feature behavior, assess class distribution, and investigate correlation patterns among chemical variables. A robust preprocessing pipeline was subsequently applied, encompassing duplicate removal, outlier elimination using the Interquartile Range (IQR) method, and feature standardization.

For the supervised learning component, multiclass classification was performed using three distinct algorithms: Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest. Among these, the Random Forest classifier achieved the highest test accuracy of 63.24% and was selected as the best-performing model. The trained model was serialized and validated through a sample prediction workflow to confirm deployment readiness.

For the unsupervised component, K-Means clustering was applied to the standardized feature space after excluding the quality label. Cluster counts ranging from $k = 2$ to $k = 6$ were evaluated using both inertia and silhouette score. The optimal solution was identified at $k = 2$, yielding a silhouette score of 0.1854, with two distinct wine groups exhibiting differing chemical profiles and quality distributions.

Overall, this project demonstrates how wine chemistry data can be leveraged to both predict quality categories with moderate success and to reveal latent groupings among wine samples through pattern discovery. The integrated pipeline—from raw data ingestion through model deployment—provides a replicable framework for structured data mining in the food science domain.

1. Introduction

1.1 Background

Wine quality is influenced by a complex interplay of measurable physicochemical properties, including acidity levels, alcohol content, residual sugar, sulphate concentrations, density, pH, and the presence of sulfur dioxide. These chemical attributes can be reliably measured in laboratory settings, making wine datasets particularly well-suited for data mining tasks that require structured, numerical input.

From a business and scientific perspective, the ability to predict wine quality from chemical composition offers significant commercial value. Wineries and distributors benefit from automated quality assessment systems that reduce reliance on time-consuming sensory evaluations by human experts. More broadly, this problem serves as an ideal case study for data warehousing and data mining, as it encompasses the full pipeline: data collection, exploratory analysis, preprocessing, predictive modeling, and pattern discovery.

This project employs the well-known red wine quality dataset to demonstrate two central goals of modern data mining: supervised prediction of categorical quality labels, and unsupervised discovery of natural groupings within the wine population. The combination of these approaches provides a holistic understanding of the dataset and showcases the complementary nature of supervised and unsupervised learning.

1.2 Problem Statement

Wine quality grading is traditionally performed by trained human tasters, a process that is inherently subjective, expensive, and difficult to scale. While laboratory measurements provide objective physicochemical data, the relationship between these measurements and perceived quality is complex and nonlinear, making simple rule-based approaches inadequate.

This project addresses two interconnected problems. First, it investigates whether wine quality categories can be accurately predicted from laboratory measurements alone, using supervised machine learning algorithms. Second, it examines whether wines can be meaningfully grouped into natural clusters based on their physicochemical properties, without relying on the quality label during model fitting. By combining both approaches in a single workflow, the project aims to provide both predictive and descriptive insights into the wine dataset.

1.3 Objectives

The main objectives of this project are:

- To conduct a thorough exploratory data analysis of the red wine quality dataset, examining feature distributions, class balance, and inter-feature correlations.
- To apply a structured preprocessing pipeline, including duplicate removal, outlier elimination, and feature normalization, to ensure data quality and model reliability.
- To develop and evaluate multiclass classification models for predicting wine quality categories, comparing Logistic Regression, K-Nearest Neighbors, and Random Forest algorithms.

2. Literature Review

2.1 Wine Quality Prediction Overview

Wine quality prediction from physicochemical data has been an active area of research since the foundational work of Cortez et al. (2009), who first released the Portuguese wine quality datasets and demonstrated that wine quality could be modeled as either a classification or regression problem. Their work established that the 11-feature chemical descriptor set (fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, and alcohol) contains sufficient information to predict human-assigned quality scores with moderate accuracy.

Since then, numerous studies have explored the same dataset using a variety of machine learning approaches, including Support Vector Machines, neural networks, gradient boosting, and deep learning architectures. A consistent finding across this body of literature is that ensemble methods — particularly Random Forest and Gradient Boosted Trees — tend to outperform linear and single-tree models, while also offering interpretable feature importance rankings that align well with domain knowledge.

2.2 Classification Algorithms

Classification is a form of supervised learning in which an algorithm learns a mapping from input features to discrete output labels using labeled training examples. In the context of wine quality prediction, the quality score (typically ranging from 3 to 8 for red wine) serves as the target class label, making this a multiclass classification problem.

Logistic Regression

Logistic Regression is a linear probabilistic classifier that models the log-odds of class membership as a linear combination of input features. When extended to multiclass settings using the softmax function (multinomial logistic regression), it provides a useful baseline for classification tasks. Its key strengths include interpretability, computational efficiency, and well-understood statistical properties. However, it assumes a linear decision boundary, which limits its capacity to capture complex nonlinear relationships common in real-world datasets.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, instance-based learning algorithm that classifies a new sample based on the majority class among its k nearest training examples, as measured by a distance metric such as Euclidean distance. KNN makes no assumptions about the underlying data distribution and can capture local patterns effectively. However, it is sensitive to the choice of k , computationally expensive at inference time for large datasets, and highly affected by the scale of input features, necessitating normalization before use.

Random Forest

Random Forest is an ensemble learning method introduced by Breiman (2001) that constructs a large number of decision trees using bootstrap sampling (bagging) and random feature subsets at each split. The final prediction is determined by majority voting across all trees. This approach significantly reduces overfitting compared to individual decision trees and improves generalization on unseen data. Random Forest also provides a natural measure of feature importance based on the average reduction in impurity contributed by each feature across all trees, offering valuable interpretability alongside strong predictive performance.

2.3 Clustering Algorithms

Clustering is an unsupervised learning technique that groups data points based on similarity in their feature space, without access to ground-truth labels. In the context of this project, clustering is used to discover natural groupings within the wine population that may not directly correspond to the quality labels assigned by human tasters.

K-Means Clustering

K-Means is one of the most widely used partitional clustering algorithms, first described by MacQueen (1967). The algorithm initializes k centroids, assigns each data point to its nearest centroid based on Euclidean distance, and then iteratively recomputes centroids as the mean of assigned points until convergence. K-Means seeks to minimize the within-cluster sum of squared distances (inertia). A key limitation is that the number of clusters k must be specified in advance. Methods such as the Elbow Method (plotting inertia against k) and the Silhouette Score are commonly used to guide this selection.

Silhouette Score

The Silhouette Score measures the cohesion and separation of clusters. For each data point, it computes the ratio of the difference between the mean intra-cluster distance and the mean nearest-cluster distance to the maximum of these two values. Scores range from -1 to +1, where values closer to +1 indicate tight, well-separated clusters. A score above 0.2 is generally considered indicative of meaningful cluster structure, though domain context is always important for interpretation.

2.4 Related Work

Cortez et al. (2009) originally framed wine quality modeling as a regression and classification problem and found that Support Vector Machines and neural networks performed comparably on the task. Subsequent work by various researchers demonstrated that ensemble methods, particularly Random Forest and Gradient Boosting, consistently achieved higher accuracy on the same dataset, with reported test accuracies in the range of 60–70% for multiclass classification on the imbalanced quality labels.

In the domain of unsupervised wine analysis, clustering has been used to discover wine typicity and to reveal hidden groupings that transcend simple quality labels. Studies using K-Means and hierarchical clustering on wine chemical data have generally found that low k values (2–4) produce the most interpretable cluster structures, with sulphates, alcohol, and acidity emerging as the most discriminating features across clusters.

The present project builds upon these findings by combining both supervised and unsupervised approaches in a single reproducible workflow, with a focus on transparent evaluation, preprocessing rigor, and practical model deployment.

3. Dataset and Methodology

3.1 Dataset Description

The project uses the red wine quality dataset (winequality-red.csv) originally compiled by Cortez et al. (2009) from the UCI Machine Learning Repository. The dataset contains physicochemical measurements for 1,599 Portuguese red wine samples of the Vinho Verde variety, along with a sensory quality score assigned by at least three wine experts on a scale of 0 (very bad) to 10 (excellent).

The dataset contains 12 columns in total: 11 physicochemical input features and one target column (quality). All features are continuous numerical variables; no categorical variables are present. The table below summarizes the key properties of the dataset at each stage of preprocessing:

Property	Value
Raw dataset rows	1,599
Total columns	12
Input feature columns	11
Target column	quality (integer, 3–8)
Missing values	0
Duplicate rows identified	240
Rows after duplicate removal	1,359
Outlier rows removed (IQR)	340
Final cleaned dataset rows	1,019

Table 3.1: Dataset summary statistics at each preprocessing stage.

Input Features

The 11 physicochemical input features used in both classification and clustering are described below:

Feature	Unit / Type	Description
Fixed Acidity	g/dm ³	Tartaric acid concentration; influences taste and stability
Volatile Acidity	g/dm ³	Acetic acid content; high levels produce a vinegar taste
Citric Acid	g/dm ³	Adds freshness and flavor; found in small quantities
Residual Sugar	g/dm ³	Sugar remaining after fermentation; affects sweetness
Chlorides	g/dm ³	Salt content; influences taste characteristics
Free Sulfur Dioxide	mg/dm ³	Free form of SO ₂ ; protects against microbial growth and oxidation
Total Sulfur Dioxide	mg/dm ³	Total SO ₂ including free and bound forms
Density	g/cm ³	Related to alcohol and sugar content
pH	Unitless	Measure of acidity or basicity; most wines fall between 3 and 4

Feature	Unit / Type	Description
Sulphates	g/dm ³	Antimicrobial and antioxidant agent; correlates with quality
Alcohol	% vol.	Percentage of alcohol content; strong positive correlation with quality

Table 3.2: Physicochemical input features in the wine quality dataset.

3.2 Tools and Environment

The project was implemented entirely in Python 3 using the Jupyter Notebook environment for interactive development and experimentation. The following libraries and tools were used:

Library / Tool	Role in Project
pandas	Data loading, manipulation, exploration, and export of CSV datasets
NumPy	Numerical computations, array operations, and statistical calculations
matplotlib	Static visualizations including histograms, bar charts, and scatter plots
seaborn	Statistical data visualization including correlation heatmaps, boxplots, and distribution plots
scikit-learn	Machine learning model development, preprocessing, cross-validation, classification, and clustering
pickle	Serialization and persistence of trained machine learning models and preprocessing objects
Jupyter Notebook	Interactive development environment for data analysis, visualization,

Library / Tool	Role in Project
	and model experimentation

Table 3.3: Libraries and tools used in the project.

3.3 Overall Pipeline

The project followed a structured, sequential pipeline designed to ensure that each stage is informed by and builds upon the findings of the preceding stage. The workflow is described below:

#	Stage	Description
1	Data Ingestion	Raw CSV dataset loaded into a pandas DataFrame; initial inspection of shape, types, and missing values
2	Exploratory Data Analysis	Statistical summaries, distribution analysis, class balance inspection, and correlation heatmap generation
3	Data Preprocessing	Duplicate removal, IQR-based outlier elimination, and feature standardization using StandardScaler
4	Classification	Training and evaluation of Logistic Regression, KNN, and Random Forest; selection and export of best model
5	Clustering	K-Means applied for $k = 2$ to 6; optimal k selected via inertia and silhouette score; PCA visualization
6	Results & Interpretation	Cluster profiles analyzed against quality labels; feature importance from Random Forest interpreted
7	Export & Persistence	Cleaned, scaled, and clustered datasets saved as CSV; best model serialized with pickle

Table 3.4: Project workflow stages.

4. Exploratory Data Analysis

4.1 Dataset Overview

Exploratory data analysis (EDA) was the first analytical step performed after loading the raw dataset. The primary objectives of EDA were to understand the data structure, identify data quality issues, examine the distribution of the target variable, and assess inter-feature relationships. All columns in the dataset are continuous numerical variables, which simplified the preprocessing requirements and confirmed the suitability of the data for both classification and clustering tasks.

Initial inspection confirmed that the dataset contains no missing values, meaning no imputation was required. However, 240 duplicate rows were identified (approximately 15% of the raw dataset), indicating that some wine samples were measured and recorded multiple times. These were flagged for removal during preprocessing.

Descriptive statistics were computed for all 11 input features and the target variable. Key observations from the statistical summary include: alcohol content ranges from approximately 8.4% to 14.9% with a mean of 10.4%; volatile acidity ranges from 0.12 to 1.58 g/dm³; and sulphate concentration ranges from 0.33 to 2.0 g/dm³. The wide ranges observed in several features, particularly sulfur dioxide measures, suggest the presence of outliers that could distort model training if left unaddressed.

4.2 Data Visualizations

Data visualization helps in understanding patterns and relationships in the dataset more effectively. Several plots were generated during EDA.

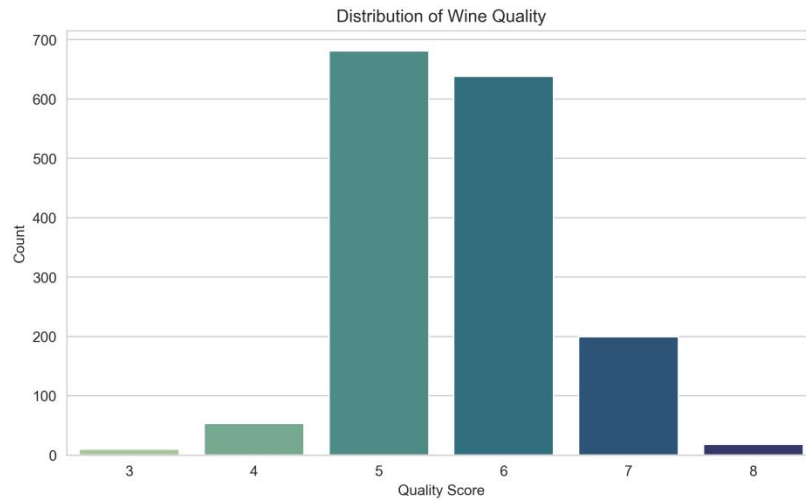


Figure 4.2.1: Target Variable Analysis

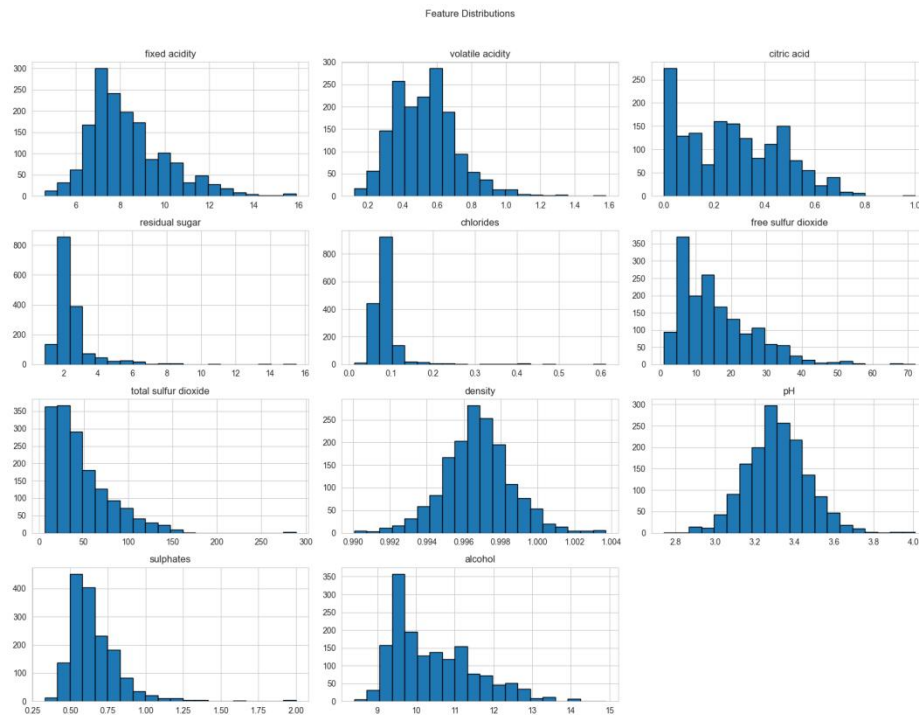


Figure 4.2.2: Univariate Analysis

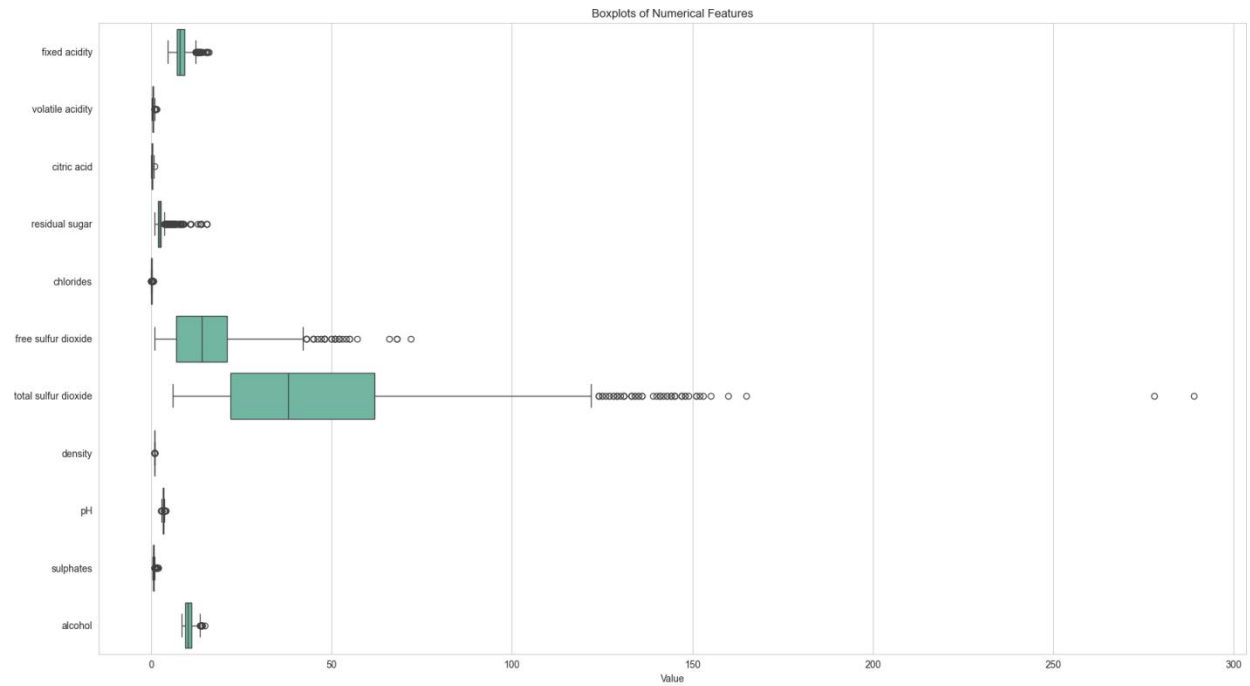


Figure 4.2.3: OutlierAnalysis

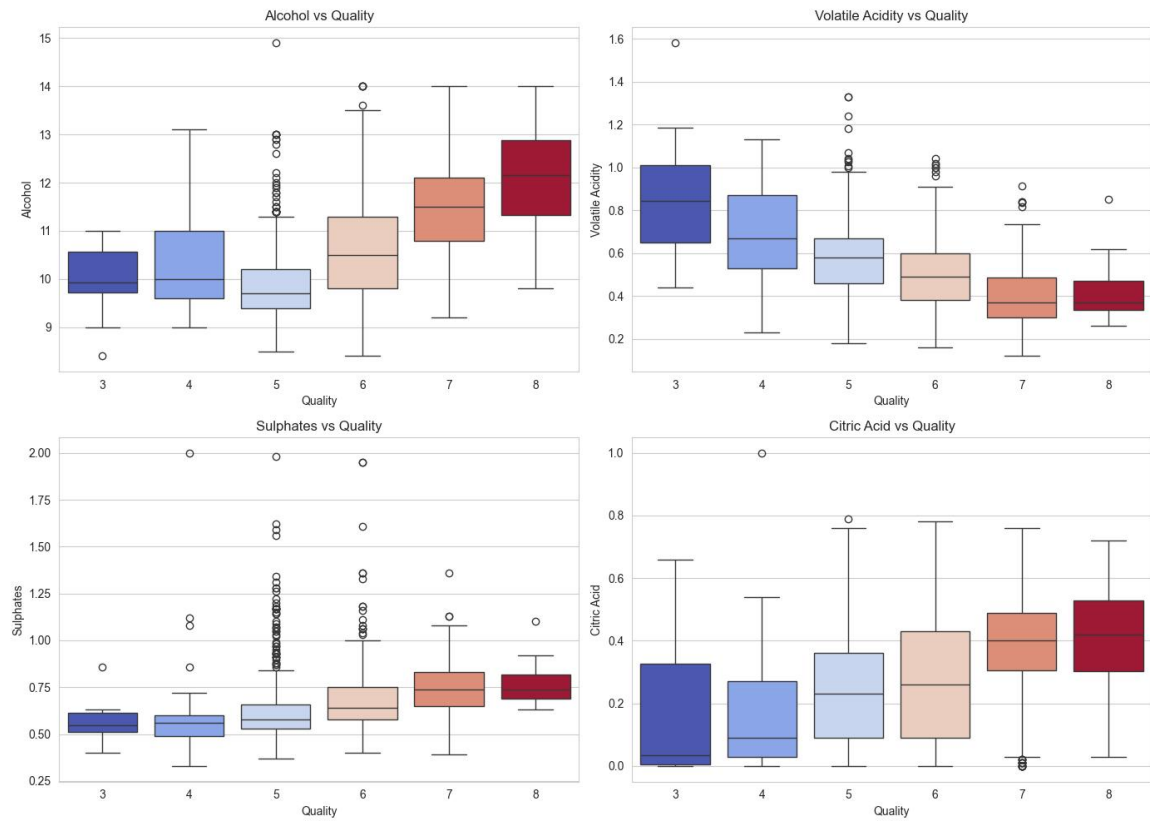


Figure 4.2.4: Feature Relationship

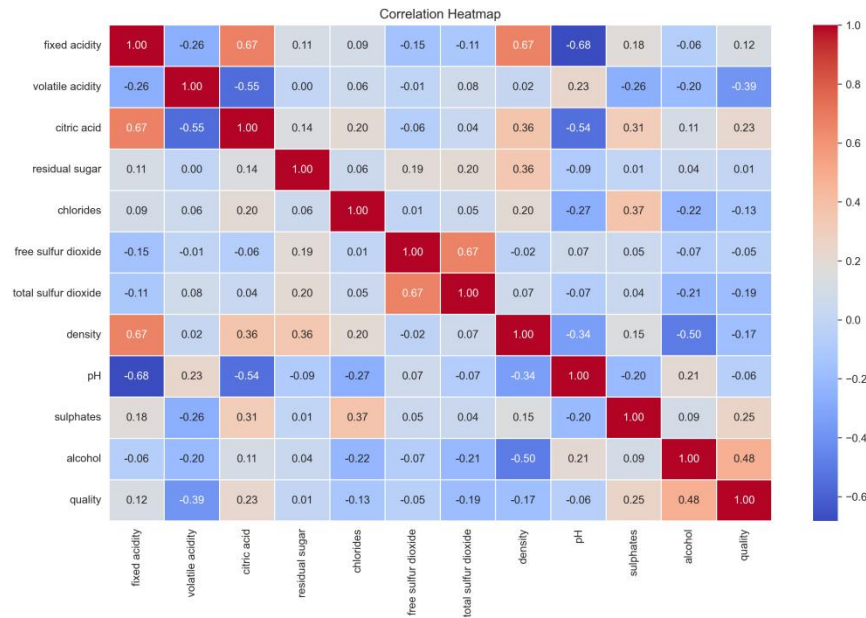


Figure 4.2.5: Pearson correlation heatmap

4.3 Key Observations and Insights

The following key observations were drawn from the exploratory data analysis phase and used to inform subsequent preprocessing and modeling decisions:

No missing values were present in the raw dataset, eliminating the need for imputation.

240 duplicate rows were identified, representing approximately 15% of the raw data. These duplicates were removed to prevent model overfitting and biased evaluation.

Several features, including residual sugar, chlorides, and sulfur dioxide measures, exhibited right-skewed distributions with notable outliers. IQR-based outlier removal was applied to address this.

The quality target variable is heavily imbalanced, with classes 5 and 6 dominating and classes 3 and 8 being very rare. Evaluation metrics beyond simple accuracy (precision, recall, F1-score) were therefore prioritized.

Alcohol, volatile acidity, and sulphates emerged as the most informative features for predicting quality, aligning with established domain knowledge in enology.

High inter-feature correlations (e.g., fixed acidity vs. density, fixed acidity vs. pH) suggest potential multicollinearity, which is less critical for tree-based models but relevant for linear methods like Logistic Regression.

5. Data Preprocessing

Data preprocessing is a critical step that transforms raw data into a clean, consistent, and appropriately scaled format suitable for machine learning. Based on the observations from EDA, the preprocessing pipeline for this project consisted of three primary stages: duplicate removal, outlier elimination, and feature standardization. Each step was carefully designed to address a specific data quality issue identified during exploratory analysis while preserving as much useful information as possible.

5.1 Handling Missing Values

First we check how many missing values exist in the dataset. Then we count and remove duplicate rows, and reset the index to keep it clean.

```
missing_values = df.isnull().sum().sum()
```

```
duplicate_rows = df.duplicated().sum()
df_no_duplicates = df.drop_duplicates().reset_index(drop=True)
```

5.2 Outlier Removal

We separate the feature columns from the target column quality. Then we calculate Q1, Q3, and IQR for each feature. Using the IQR rule, we create a mask to filter out any row where even one feature falls outside the bounds. Only clean rows are kept.

```
feature_columns = [col for col in df_no_duplicates.columns if col != 'quality']
Q1 = df_no_duplicates[feature_columns].quantile(0.25)
Q3 = df_no_duplicates[feature_columns].quantile(0.75)
IQR = Q3 - Q1
mask = ~((df_no_duplicates[feature_columns] < (Q1 - 1.5 * IQR)) |
(df_no_duplicates[feature_columns] > (Q3 + 1.5 * IQR))).any(axis=1)
df_clean = df_no_duplicates[mask].reset_index(drop=True)
```

5.3 Feature Scaling

We split the data into features X and target y. Then we apply StandardScaler to standardize all features to zero mean and unit variance. The scaled values are put back into a DataFrame with the original column names and quality column added back.

Code:

```
X = df_clean.drop(columns='quality')
y = df_clean['quality']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
df_scaled = pd.DataFrame(X_scaled, columns=X.columns)
df_scaled['quality'] = y.values
```

5.4 Processed Dataset

Saving Outputs:

We save two files — the cleaned unscaled data as `winequality_cleaned.csv` and the scaled data as `winequality_scaled.csv` — to the processed folder for use in modeling.

```
cleaned_output_path = processed_dir / 'winequality_cleaned.csv'
```

```
scaled_output_path = processed_dir / 'winequality_scaled.csv'
```

```
df_clean.to_csv(cleaned_output_path, index=False)
```

```
df_scaled.to_csv(scaled_output_path, index=False)
```

6. Clustering

6.1 K-Means Algorithm Overview

K-Means is an unsupervised machine learning algorithm used for grouping similar data points into K distinct clusters based on feature similarity. It is widely used for customer segmentation due to its simplicity and efficiency. The algorithm begins by randomly initializing K centroids. Each data point is then assigned to the nearest centroid based on a distance metric, typically Euclidean distance. After assignment, the centroids are recalculated as the mean of all data points within each cluster. This process is repeated iteratively until the centroids no longer change significantly, indicating convergence.

The main objective of K-Means is to minimize the within-cluster variation, ensuring that data points within the same cluster are as similar as possible while maximizing separation between different clusters.

K-Means Objective Function

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2,$$

Where:

- k represents the number of clusters
- C_i represents the data points in cluster i
- μ_i represents the centroid of cluster i

Code:

```
k_optimal = 3
```

```
kmeans = KMeans(n_clusters=k_optimal, random_state=42, n_init=10)
```

```
clusters = kmeans.fit_predict(df_features)
```

6.2 Optimal K Selection

Metrics Used

1. Elbow Method

- Plots the inertia (within-cluster sum of squares) against the number of clusters
- Identifies the "elbow" point where adding more clusters provides diminishing returns
- Helps determine the optimal balance between model complexity and performance

2. Silhouette Score

- Measures how similar an object is to its own cluster compared to other clusters
- Range: [-1, 1]
- Higher values indicate well-separated clusters
- Values closer to 1 indicate strong cluster cohesion

Range Tested: $k = 2$ to 6 clusters

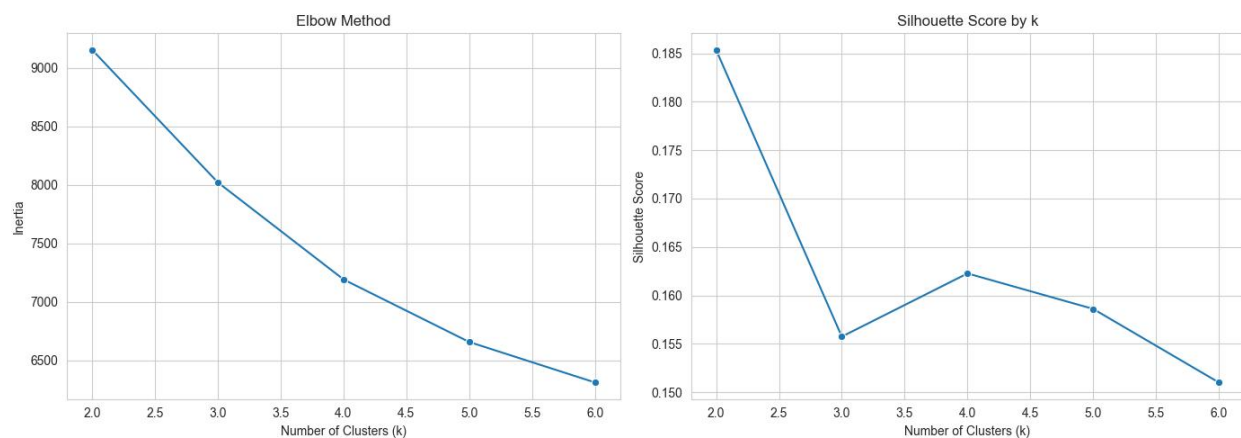


Figure 6.2 : Elbow Method and Silhouette Score

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.lineplot(data=results_df, x='k', y='inertia', marker='o', ax=axes[0])
axes[0].set_title('Elbow Method')
axes[0].set_xlabel('Number of Clusters (k)')
axes[0].set_ylabel('Inertia')
sns.lineplot(data=results_df, x='k', y='silhouette_score', marker='o', ax=axes[1])
axes[1].set_title('Silhouette Score by k')
```

```

axes[1].set_xlabel('Number of Clusters (k)')
axes[1].set_ylabel('Silhouette Score')
plt.tight_layout()
plt.savefig(plot_dir / 'cluster_selection_metrics.png', dpi=300, bbox_inches='tight')
plt.show()

```

6.3 Cluster Visualization

Each point represents a wine sample

- Colors represent different clusters
- Spatial proximity indicates similarity in chemical composition
- Well-separated clusters in PCA space indicate robust clustering

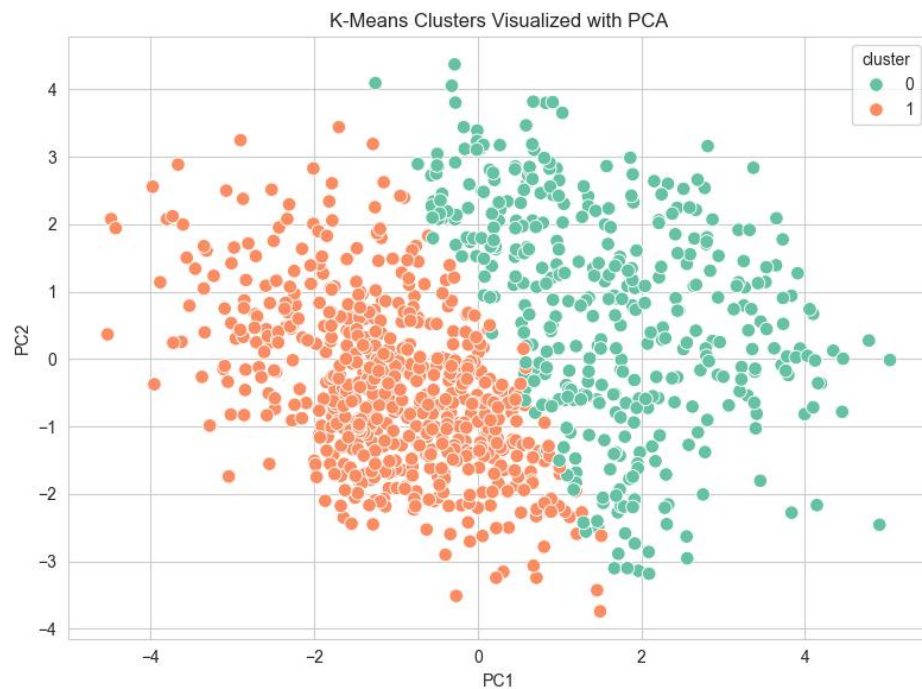


Figure 6.3 : Cluster visualization

6.4 Cluster Interpretation

Visual representation confirms cluster separation quality

- PCA components capture the main drivers of cluster differences
- Visualization validates the silhouette score metrics

```
cluster_counts = df_clustered['cluster'].value_counts().sort_index()
display(cluster_counts.to_frame(name='count'))
```

count	
cluster	
0	401
1	618

```
quality_cluster_table = pd.crosstab(df_clustered['cluster'], df_clustered['quality'])
display(quality_cluster_table)
```

quality	3	4	5	6	7	8
cluster						
0	1	7	122	178	84	9
1	2	26	312	245	31	2

```
cluster_profile = df_clustered.groupby('cluster').mean(numeric_only=True)
display(cluster_profile)
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
cluster												
0	9.459352	0.402756	0.425860	2.321945	0.079504	14.137157	39.249377	0.997149	3.242020	0.684539	10.680133	5.907731
1	7.333172	0.600210	0.137217	2.118932	0.077770	15.493528	44.425566	0.996156	3.374531	0.596650	10.195092	5.457929

Cluster Descriptions

Cluster 0: Higher-quality chemical profile segment

Wines in this cluster show relatively higher average values for fixed acidity, citric acid, sulphates, alcohol, and overall quality. This cluster also contains a higher proportion of quality 6, 7, and 8 wines compared to the other cluster. Based on these characteristics, Cluster 0 can be interpreted as the group of wines with stronger chemical profiles and comparatively better quality tendency.

Cluster 1: Average-quality wine segment

Wines in this cluster are more concentrated around quality category 5 and contain a larger number of average-quality wines. Compared to Cluster 0, this cluster shows lower average alcohol, citric acid, sulphates, fixed acidity, and overall quality. Therefore, Cluster 1 can be interpreted as the group of wines with more moderate chemical characteristics and average quality tendency.

6.5 Evaluation

The performance and quality of the K-Means clustering model were evaluated using Inertia and Silhouette Score. These metrics were used to compare different values of K and select the most suitable number of clusters for the wine dataset.

Inertia measures the total within-cluster variation. A lower inertia value indicates that data points are closer to their assigned cluster centers. In this project, inertia decreased as the number of clusters increased, which is expected in K-Means clustering. However, inertia alone is not enough to select the best K, so it was used together with the silhouette score.

```
inertia = kmeans.inertia_
```

Silhouette Score measures how similar each data point is to its own cluster compared to other clusters. It ranges from -1 to 1, where a higher value indicates better cluster separation. In this project, the highest silhouette score was obtained when $K = 2$. The silhouette score for $K = 2$ was 0.1854, which indicates weak to moderate cluster separation. This means the clusters show some meaningful grouping, but there is still overlap between them.

```
silhouette = silhouette_score(X_scaled, cluster_labels)
```

Based on the comparison of inertia and silhouette score, $K = 2$ was selected as the final number of clusters. Although the silhouette score is not very high, the result is acceptable for exploratory clustering because the dataset contains continuous chemical measurements and the quality classes are naturally close to each other.

K	Inertia	Silhouette Score	Interpretation
2	9151.27	0.1854	Best silhouette score among all tested values
3	8020.08	0.1557	Lower silhouette score
4	7191.86	0.1623	Slight improvement but still below $K = 2$
5	6658.01	0.1586	Lower silhouette score
6	6312.77	0.1510	Lowest silhouette score among all tested values

Table 6.2: Cluster Selection Results for Different K Values

7. Classification

7.1 Classification Objective

The primary goal of the classification component is to develop a predictive model capable of assigning a wine quality category based solely on its 11 physicochemical input features. Because the quality column contains discrete integer values (ranging from 3 to 8 in this dataset), this is a multiclass classification problem rather than a binary or regression problem.

Accurate quality prediction from chemical measurements has practical applications in automated quality control systems, where rapid and objective assessment is preferred over labor-intensive expert tasting. The classification task was designed to be as rigorous as possible, including proper train-test splitting, stratification, feature scaling within the classification workflow, and multi-metric evaluation.

7.2 Methodology

7.2.1 Data Preparation

The dataset used for classification is `'winequality_cleaned.csv'`, which contains all preprocessing and cleaning completed in previous phases. The data is fully ready for supervised learning with labeled quality targets, comprising 11 chemical features as input variables. Feature-target separation was performed by assigning all 11 chemical properties of wine to the feature matrix X and the wine quality rating, ranging from 3 to 8, as the target variable y . The quality distribution across the dataset reflects imbalanced classes, with a higher concentration of mid-range quality wines compared to extreme quality levels.

7.2.2 Train-Test Split

The dataset was divided using a stratified random split strategy, allocating 80% of the data for model training and reserving 20% as a held-out test set for unbiased evaluation. Stratification was applied to ensure that the class distribution of quality levels is maintained proportionally in both the training and test sets, preventing any quality category from being over- or under-represented in either subset. A fixed random state of 42 was set to guarantee reproducibility of the split across runs. This approach ensures balanced representation of all quality levels, prevents bias in model evaluation, and allows reliable accuracy estimation on unseen data.

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

7.2.3 Feature Scaling

Feature scaling was applied using StandardScaler, which standardizes each feature to zero mean and unit variance. Scaling was applied selectively based on the requirements of each algorithm. Logistic Regression and K-Nearest Neighbors both received scaled features, as Logistic Regression relies on gradient-based optimization and KNN uses distance calculations, both of which are sensitive to the magnitude of input features. Random Forest was trained on unscaled features, as it is a tree-based algorithm that makes split decisions based on ranked order rather than absolute values, making it inherently scale-invariant.

```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

7.2.4 Model Training and Evaluation

Three classification models are trained and compared:

- Logistic Regression
- K-Nearest Neighbors
- Random Forest

```

models = {
    'Logistic Regression': (LogisticRegression(max_iter=1000), X_train_scaled, X_test_scaled),
    'K-Nearest Neighbors': (KNeighborsClassifier(n_neighbors=5), X_train_scaled,
X_test_scaled),
    'Random Forest': (RandomForestClassifier(n_estimators=200, random_state=42), X_train,
X_test)
}

results = []

trained_models = {}

for name, (model, X_train_input, X_test_input) in models.items():
    model.fit(X_train_input, y_train)
    y_pred = model.predict(X_test_input)
    accuracy = accuracy_score(y_test, y_pred)
    results.append({'Model': name, 'Accuracy': accuracy})
    trained_models[name] = {'model': model, 'y_pred': y_pred}

results_df=pd.DataFrame(results).sort_values(by='Accuracy',
ascending=False).reset_index(drop=True)

display(results_df)

```

7.3 Model Comparison

Three classifiers representing different learning paradigms were trained and evaluated on the same train-test split to allow a fair comparison. Each model was trained with default hyperparameters as a baseline, with a fixed random state where applicable to ensure reproducibility.

Model	Learning Paradigm	Test Accuracy
Logistic Regression	Linear, Probabilistic	59.80%
K-Nearest Neighbors	Instance-Based	54.90%
Random Forest	Ensemble, Tree-Based	63.24%

Table 7.1: Classification model comparison by test accuracy.

Random Forest achieved the highest test accuracy at 63.24%, followed by Logistic Regression at 59.80% and K-Nearest Neighbors at 54.90%. The superiority of Random Forest is consistent with the established literature on this dataset, reflecting its ability to capture nonlinear feature interactions and its robustness to the class imbalance through ensemble averaging.

The relatively lower performance of KNN compared to Logistic Regression is noteworthy and likely reflects the curse of dimensionality in an 11-dimensional feature space, where distance-based methods struggle without careful hyperparameter tuning.

7.4 Best Model Evaluation

Random Forest was selected as the best-performing model and subjected to comprehensive evaluation. The full evaluation included test set accuracy, a detailed classification report (precision, recall, and F1-score per class), and a confusion matrix.

The weighted F1-score achieved by the best model was approximately 0.607, reflecting the model's overall performance across all classes weighted by class support. The classification report revealed a clear pattern consistent with the class imbalance: precision, recall, and F1-score were highest for quality 5 and 6 (the dominant classes), while performance on quality 3, 4, 7, and 8 was considerably lower, particularly for recall.

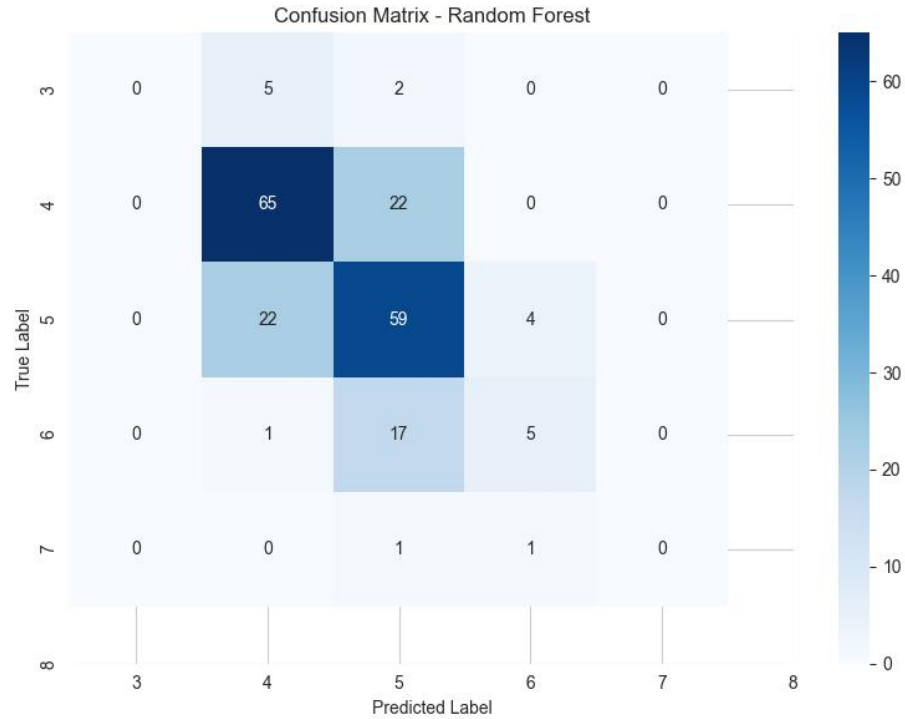


Figure 7.1: Confusion matrix of the best-performing Random Forest classifier on the test set.

The confusion matrix further illustrates this pattern. The model correctly classified the majority of quality-5 and quality-6 samples, with most misclassifications occurring in adjacent quality classes (e.g., predicting 5 when the true label is 6). Rare extreme classes (3 and 8) were rarely predicted correctly, with the model tending to assign these samples to the adjacent majority class. This behavior is expected and consistent with the class imbalance: the model has far more training examples for the middle classes and therefore learns their decision boundaries more reliably. Future work could address this through oversampling techniques (e.g., SMOTE), class weighting, or specialized ensemble methods designed for imbalanced data.

7.5 Feature Importance Analysis

Random Forest provides a built-in measure of feature importance based on the mean decrease in impurity (Gini importance) across all trees in the ensemble. Feature importance analysis was performed to identify which physicochemical properties most strongly drive quality predictions.

The results indicate that alcohol is by far the most important feature for predicting wine quality, consistent with the strong positive correlation observed during EDA. The top-ranked features in descending order of importance are:

- Alcohol: Most influential feature; wines with higher alcohol tend to achieve higher quality scores.
- Sulphates: Second most important; higher concentrations associated with better quality.
- Volatile Acidity: Strong negative predictor; high acetic acid content degrades perceived quality.
- Total Sulfur Dioxide: Moderate importance; excessively high levels negatively affect taste.
- Citric Acid: Mild positive predictor; contributes to freshness and flavor complexity.

Features such as chlorides, residual sugar, and free sulfur dioxide ranked at the lower end of the importance spectrum, suggesting they contribute less to quality differentiation in this dataset. These findings are largely consistent with established enological knowledge and support the interpretability of the Random Forest model.

7.6 Model Persistence and Sample Prediction

The best-performing Random Forest classifier was serialized to disk using Python's pickle library and saved as `best_classification_model.pkl` in the project's models directory. Model persistence is an essential step for practical deployment, enabling the trained model to be loaded and used for inference without retraining.

To validate the serialization process, the saved model was reloaded from disk and re-evaluated on the test set. The loaded model produced identical accuracy results (63.24%), confirming that serialization and deserialization preserved model state correctly.

A sample prediction was then performed by selecting a single wine observation from the test set. The actual quality label for this sample was 5, and the reloaded model correctly predicted a quality of 5, demonstrating end-to-end functionality from model loading through prediction output.

```
sample_index = X_test.index[0]
sample_features = X_test.loc[[sample_index]]
actual_quality = y_test.loc[sample_index]
if loaded_scaler is not None:
    sample_input = loaded_scaler.transform(sample_features)
else:
    sample_input = sample_features
predicted_quality = loaded_model.predict(sample_input)[0]
```

Sample wine features:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
652	7.4	0.965	0.0	2.2	0.088	16.0	32.0	0.99756	3.58	0.67	10.2

Actual quality: 5
Predicted quality: 5

8. Discussion

8.1 Limitations

Despite achieving reasonable performance, the model has several limitations:

- The dataset is imbalanced, with most wines belonging to quality categories 5 and 6.
- Rare quality classes such as 3 and 8 have very few samples, so classification performance on those classes is weak.
- Outlier removal reduced the dataset size, which may have removed some unusual but valid wine samples.
- Only three classification models were compared: Logistic Regression, KNN, and Random Forest.
- Only K-Means clustering was used for clustering.
- The silhouette score was low, meaning the clusters have some overlap and are not strongly separated.
- The project used only physicochemical features and did not include sensory or expert tasting notes.

8.2 Future Improvements

To improve the system in future work, the following enhancements can be considered:

- Apply class-balancing techniques such as oversampling, undersampling, or class weights.
- Perform hyperparameter tuning to improve classification model performance.
- Compare more classification algorithms such as SVM, Gradient Boosting, or XGBoost.
- Try additional clustering methods such as Hierarchical Clustering or DBSCAN.
- Use feature selection or feature engineering to improve model interpretability.
- Compare results before and after outlier removal.
- Add feature importance analysis to better understand which chemical properties affect wine quality most.
- Use a larger or more balanced wine dataset if available.

9. Conclusion

This project successfully applied data mining techniques to the red wine quality dataset. Exploratory Data Analysis helped identify important characteristics of the dataset, including class imbalance, duplicate records, outliers, and relationships among physicochemical features. Preprocessing was then performed by removing duplicates, handling outliers using the IQR method, and applying feature scaling where needed.

For category prediction, three classification models were compared, and Random Forest achieved the best performance among the selected models. For clustering, K-Means was used to group wines based on chemical properties, and $K = 2$ was selected using silhouette score and inertia. Overall, the project shows that wine quality can be predicted with moderate accuracy and that wines can be grouped into meaningful but partially overlapping clusters.

10. References

- [1] Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. *Wine Quality Dataset and Modeling Study*.
- [2] Han, J., Kamber, M., & Pei, J. *Data Mining: Concepts and Techniques*.
- [3] Tan, P. N., Steinbach, M., & Kumar, V. *Introduction to Data Mining*.
- [4] scikit-learn Documentation. *Classification, Clustering, and Evaluation Metrics*. Available at: <https://scikit-learn.org>
- [5] pandas Documentation. *Data Cleaning and Data Analysis*. Available at: <https://pandas.pydata.org>
- [6] seaborn Documentation. *Statistical Data Visualization*. Available at: <https://seaborn.pydata.org>
- [7] matplotlib Documentation. *Python Plotting Library*. Available at: <https://matplotlib.org>

11. Appendices

File Generated	Notebook / Source	Description
winequality_cleaned.csv	02_Preprocessing.ipynb	Cleaned dataset after duplicate removal and outlier handling
winequality_scaled.csv	02_Preprocessing.ipynb	Scaled version of the cleaned dataset using StandardScaler
winequality_clustered.csv	04_Clustering.ipynb	Dataset with assigned K-Means cluster labels
quality_distribution.png	01_EDA.ipynb	Bar chart showing distribution of wine quality categories
feature_distributions.png	01_EDA.ipynb	Histograms showing distribution of all numerical features
feature_boxplots.png	01_EDA.ipynb	Boxplots used to observe outliers in numerical features
feature_vs_quality_boxplots.png	01_EDA.ipynb	Boxplots showing selected feature relationships with quality
correlation_heatmap.png	01_EDA.ipynb	Heatmap showing correlation between wine features and quality
confusion_matrix_best_model.png	03_Classification.ipynb	Confusion matrix of the best classification model
cluster_selection_metrics.png	04_Clustering.ipynb	Elbow and silhouette score plots for selecting number of

File Generated	Notebook / Source	Description
		clusters
kmeans_pca_clusters.png	04_Clustering.ipynb	PCA visualization of final K-Means clusters
best_classification_model.pkl	03_Classification.ipynb	Saved best classification model for reuse and testing
01_EDA.ipynb	Notebook	Exploratory Data Analysis notebook
02_Preprocessing.ipynb	Notebook	Data preprocessing notebook
03_Classification.ipynb	Notebook	Classification and model saving notebook
04_Clustering.ipynb	Notebook	K-Means clustering notebook
Readme.md	Project root	Project overview, workflow, outputs, and instructions
wine-quality-mining-and-category-prediction.docx	Document folder	Final project documentation report